HOSTEL OS: An Intelligent Hostel Operations,Safety and Resource Optimization Platform

B.TECH PHASE-I PROJECT REPORT

Submitted by

GOKUL SR (BMC23CS056)

JEEN GEORGE MATHUNNY (BMC23CS057)

S ADITHYAN (BMC23CS094)

ALBI THOMAS (LBMC23CS105)

Under the guidance of

Prof. MANOLEKSHMI I

(Assistant Professor, Department of Computer Science)

To

the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the Degree

of

BACHELOR OF TECHNOLOGY IN

COMPUTER SCIENCE AND ENGINEERING



Department of Computer Science and Engineering

BASELIOS MATHEWS II COLLEGE OF ENGINEERING

(Approved by AICTE & Affiliated to A.P.J ABDUL KALAM TECHNOLOGICAL UNIVERSITY)

Lake View, Muthupilakkadu, Sasthamcotta - 690 520, Kollam, Kerala

OCTOBER 2026



BASELIOS MATHEWS II COLLEGE OF ENGINEERING

Department of Computer Science and Engineering

An Intelligent Hostel Operations,Safety and Resource Optimization Platform

HOSTEL OS

A Project Report submitted to

APJ Abdul Kalam Technological University

In partial fulfillment of the requirements for the award of the degree of

Bachelor of technology in Computer Science and Engineering

Project Guide	**:**	**Asst. Prof. Manolekshmi I**
Project Coordinator	**:**	**Dr. Jayakrishnan R**

Gokul S R	**BMC23CS056**
Jeen George Mathunny	**BMC23CS057**
S Adithyan	**BMC23CS094**
Albi Thomas	**LBMC23CS105**

October 2026

DECLARATION

We undersigned hereby declare that the phase-1 project report titled – "HOSTEL OS: An Intelligent Hostel Operations,Safety and Resource Optimization Platform", submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by us under supervision of Prof. Manolekshmi I. This submission represents our ideas in our own words and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

ALBI THOMAS

GOKUL S R

S ADITHYAN

JEEN GEORGE MATHUNNY

Place : Sasthamcotta

Date :

BASELIOS MATHEWS II COLLEGE OF ENGINEERING

Lake view, Muthupilakadu, Sasthamotta – 690520,Kollam,Kerala

CERTIFICATE

This is to certify that the report entitled "HOSTEL OS: An Intelligent Hostel Operations,Safety and Resource Optimization Platform"*submitted by ALBI THOMAS (LBMC23CS105), GOKUL S R (BMC23CS056), JEEN GEORGE MATHUNNY (BMC23CS057), S ADITHYAN (BMC23CS094) to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements of the BTech Degree in Computer Science Engineering is a bonafide record of the project phase-1 work carried out by them under my guidance and super vision. This report in any form has not been submitted to any other University or Institute for any purpose.*

Prof. MANOLEKSHMI I
Assistant professor
(Project Guide)
Department of Computer Science & Engineering
Baselios Mathews II College Of Engineering

Prof. Dr JAYAKRISHNAN R
Associate Professor
(Project Coordinator)
Department of Computer Science & Engineering
Baselios Mathews II College Of Engineering

Prof. GIMY JOY
HEAD OF THE DEPARTMENT
Department of Computer Science & Engineering,
Baselios Mathews II College of Engineering

Internal Examiner **External Examiner**

ACKNOWLEDGEMENT

At the outset, we find it our obligation to thank the Almighty God for giving us necessary wisdom to accomplish this project phase-1 report. We are extremely grateful to the management for providing adequate infrastructural amenities to work in, without which this work would not have been feasible.

We would like to express our heartfelt thankfulness with at-most esteem and gratitude to the principal **Prof Dr. L Padma Suresh** for his opportune to help and permitting us to utilize the facilities for the successful completion of our project and the preparation of this report.

We are privileged to place on the record our profound sense of gratitude and appreciation to **Prof. Gimy Joy**, Head of the Department, Department of Computer Science & Engineering, for her generous guidance, help and constructive suggestions.

We express our profound gratitude to **Prof. Manolekshmi I,** project guide for her advice, supervision and patience throughout the work.

And also we express our profound gratitude to **Prof. Dr Jayakrishnan R,** project coordinator for his valuable guidance and for giving us this wonderful opportunity to widen our knowledge in this field of Computer Science. We also express our heartfelt thanks to all our friends, classmates and our parents for providing the much needed support and assistance in the preparations for this project report.

We would like to thank everyone, who had been pillars of support and guidance tome throughout the course of work, taking it present form.

ALBI THOMAS
GOKUL S R
S ADITHYAN
JEEN GEORGE MATHUNNY

ABSTRACT

An intelligent and unified hostel management platform is proposed to improve the efficiency, safety, and resource utilization of hostel operations. Conventional hostel management systems primarily focus on administrative activities and often operate as fragmented modules with limited intelligence and decision-support capabilities. The proposed system addresses this limitation by integrating hostel administration with artificial intelligence, predictive analytics, real-time monitoring, and resource optimization. The proposed system provides centralized management of students, rooms and beds, leave requests, entry and exit movements, fees, complaints, mess operations, inventory, maintenance, visitors, and emergency incidents. Rolebased access control enables secure interaction for students, parents or guardians, wardens, security personnel, administrators, and mess or maintenance staff. The platform incorporates AI-based complaint classification, explainable room allocation, predictive maintenance, mealdemand forecasting, inventory forecasting, and risk analysis. An emergency management module supports SOS reporting, alerts, acknowledgement, tracking, and audit logging. The primary objective is to transform hostel management from basic record keeping into intelligent monitoring, prediction, optimization, and decision support. The proposed system is expected to improve operational efficiency, resource utilization, safety, complaint handling, meal planning, maintenance management, and administrative decision-making.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	ACKNOWLEDGEMENT	**i**
	ABSTRACT	**ii**
	TABLE OF CONTENTS	**iii**
	LIST OF FIGURES	**vi**
	LIST OF TABLES	**vii**
	LIST OF ABBREVATIONS	**viii**
1	**CHAPTER 1: INTRODUCTION**	**1**
	1.1 Background of the study	2
	1.2 Problem Statement	3
	1.3 Objectives	4
	1.4 Scope of the Project	4
	1.5 Significance of the Project	5
	1.6 Contributions	6
	1.7 Organization of the Project	7
2	**CHAPTER 2: LITERATURE REVIEW**	**8**
	2.1 Introduction	8
	2.2 Hostel Management and Intelligent Campus System	8
	2.3 Review of Related Research	9
	2.3.1 QR Code-based Access Control Systems	9
	2.3.2 Smart Hostel Management Through the IoT	10
	2.3.3 Smart lock system using QR code technology	10
	2.3.4 Digitizes Hostel Administrative workflows	10
	2.3.5 Smart door access control system based on QR	11
	2.3.6 Hostel management system using artificial intelligence and IoT	11
	2.3.7 IoT-based smart hostel management system	11

Chapter No.	Title	Page No.
	2.3.8 Hostel management system with QR codes	12
3	**CHAPTER 3: SYSTEM ANALYSIS AND DESIGN**	**13**
	3.1 Introduction	13
	3.2 Existing System	13
	3.3 Proposed System	15
	3.4 Functionalities	15
	3.5 Non-Functional Requirements	16
	3.6 System Architecture	17
	3.7 User roles and access model	18
	3.8 Gate Authorization Model	19
	3.9 User Workflow examples	19
	3.10 Security and Privacy Design	21
	3.11 UML-Oriented system views	22
	3.11.1 Use case view	22
	3.11.2 Sequence view	23
	3.12 Data Architecture	25
	3.13 Data Elements	27
	3.14 Threat considerations	27
4	**CHAPTER 4: IMPLEMENTATION**	**29**
	4.1 Introduction	29
	4.2 Software Requirements	29
	4.3 Hardware Requirements	31
	4.4 System Modules	32
	4.4.1 Core Administration,Identity and Movement Authorization Module	32
	4.4.2 Safety,Emergency and Gate Verification Module	34
	4.4.3 Intelligence and Analytics Module	36
	4.4.4 Resident Services and Finance Module	38

Chapter No.	Title	Page No.
	4.4.5 Module Allocation among Team members	40
5	**CHAPTER 5: CONCLUSION**	**41**
	5.1 Conclusion	41
	5.2 Future Scope	42
	REFERENCES	**43**

LIST OF FIGURES

Figure No.	Description	Page No.
3.1	High-Level HostelOS System Architecture	17
3.2	Simplified Use Case View	22
3.3	Sequence for Exceptional-Movement QR Authorization	24
3.4	Conceptual Entity -Relationship Diagram of HostelOS	26
4.1	Exceptional-Movement Request and Token Generation	34
4.2	Server-Side QR Verification Workflow	35
4.3	Intelligence and Analytics Workflow	38
4.4	Resident Services and Finance Workflow	40

LIST OF TABLES

Table No.	Description	Page No.
3.1	Roles and Access Scope	18
3.2	Major Data Entities and Attributes	27
4.1	Proposed Software Requirements of HostelOS	31
4.2	Hardware Requirements of HostelOS	31
4.3	Proposed Module Allocation	40

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CCTV	Closed-Circuit Television
CPU	Central Processing Unit
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
IoT	Internet of Things
iOS	iPhone Operating System
JWT	JSON Web Token
LAN	Local Area Network
ML	Machine Learning
NFC	Near Field Communication
NLP	Natural Language Processing
OS	Operating System
PC	Personal Computer
PIR	Passive Infrared
QR	Quick Response
RAM	Random Access Memory
RBAC	Radio-Based Access Control
SOS	Emergency Distress Signal
SSD	Solid-State Drive
UI	User Interface
vCPU	Virtual Central Processing Unit
Wi-Fi	Wireless Fidelity

CHAPTER 1

INTRODUCTION

A hostel is a shared residential environment where many students live, study, eat and move within common infrastructure. Running it well means coordinating room and bed allocation, leave and attendance, entry and exit movement, fee collection, complaints, mess operations, inventory, maintenance, visitors and emergencies. Each of these involves different stakeholders, including students, parents or guardians, wardens, security personnel, administrators and service staff. Most hostel software in use today only digitizes forms and records. Its modules are built in isolation, and the data they collect is seldom used for prediction, automation or decision support.

Existing research has improved individual parts of hostel operations through RFID and biometric attendance, IoT-based energy and safety monitoring, facial-recognition-based visitor detection, QR-based access control, and machine-learning-based complaint classification. However, these solutions are usually hardware-dependent or limited to a single domain. Access control in particular often relies on dedicated readers or locks, which add cost and daily friction even for routine, permitted movement. A review of the literature shows that no existing system combines administration, safety, predictive analytics and resource optimization on one platform for hostels.

This project proposes HostelOS, an intelligent and unified hostel operations, safety and resource optimization platform. It centralizes the management of students, rooms and beds, leave, fees, complaints, mess, inventory, maintenance, visitors and emergencies, with role-based access for each type of user. On top of this, it adds an AI and analytics layer that provides complaint classification and priority routing, explainable room allocation, predictive maintenance, meal-demand and inventory forecasting, and risk analysis. Exceptional movement, such as late returns, holiday outings and medical visits, is authorized through staff-approved temporary QR tokens

that are verified and logged on the server, so no dedicated access-control hardware is needed and routine movement stays free of friction. An emergency module supports SOS reporting, alerts, acknowledgement tracking and audit logging.

The primary aim of HostelOS is to move hostel management from basic record keeping to intelligent monitoring, prediction, optimization and decision support. The platform is expected to improve operational efficiency, resource utilization, safety, complaint handling, meal planning and maintenance management, and to give wardens and administrators dependable, data-driven support for planning and decision-making.

1.1 BACKGROUND OF THE STUDY

A hostel is a residential environment in which a large number of students live, study, eat and move within a shared infrastructure. Running it well means coordinating many activities at once: allocating rooms and beds, tracking attendance and leave, controlling entry and exit, collecting fees, handling complaints, planning mess operations, maintaining inventory and facilities, managing visitors, and responding to emergencies. Each of these activities involves different people, including students, parents or guardians, wardens, security personnel, administrators, and mess and maintenance staff. All of them depend on timely and accurate information.

Early hostel management software focused on replacing paper registers with digital records. Such systems store student details, room assignments and payments, and they reduce manual paperwork [6]. Later research introduced technologies such as RFID and biometric verification for entry and attendance [1], IoT sensor networks for energy and safety monitoring [3][8], facial-recognition cameras for visitor detection [4], and QR code-based access control [10]. These contributions improved individual aspects of hostel operations, but most of them address a single function, often at the cost of dedicated hardware at every access point.

At the same time, machine learning, predictive analytics and natural language processing have matured to the point where they can classify text, forecast demand and estimate risk from historical data. Complaint classification using ML and NLP has been demonstrated in other domains [9], and time-series and regression techniques are widely used for demand forecasting. A hostel generates exactly the

kind of data these techniques need (complaint records, maintenance histories, meal consumption and stock usage), yet this data is rarely used for prediction or decision support.

This project, **HostelOS**, is motivated by that gap. It proposes a unified hostel platform that combines day-to-day administration with safety management, artificial intelligence and resource optimization, and that treats the hostel as a system to be monitored and optimized rather than merely recorded.

1.2 PROBLEM STATEMENT

Conventional hostel management systems mainly digitize administrative activity. They are usually built as isolated modules for fees, complaints, mess, rooms and attendance, with little data sharing between them. As a result, the data they collect is seldom used for prediction, automation or decision support, and wardens and administrators continue to react to problems after they occur rather than anticipating them.

Access control raises a further difficulty. Existing approaches commonly depend on additional hardware such as RFID tags, biometric readers or smart-lock units. These add procurement and maintenance cost, and they add daily friction, since even routine, permitted movement must be verified at the gate. QR-based systems reduce hardware needs but are often used for routine daily attendance, which again burdens ordinary movement. They are also typically disconnected from student leave status and from the wider hostel administration [2][5].

Finally, the review of related work shows that no single existing system combines administration, safety, predictive analytics and resource optimization for hostels. The research gap therefore lies between basic hostel management software and a unified intelligent platform. This project addresses the question of how such a platform can be designed so that it digitizes hostel operations end to end, keeps exceptional movement authorized and traceable without dedicated access-control hardware, and uses AI-based analysis to support complaint handling, maintenance, meal planning, inventory management and emergency response.

1.3 OBJECTIVES

The primary objective of the project is to transform hostel management from basic record keeping into intelligent monitoring, prediction, optimization and decision support. The specific objectives are:

- To digitize role-based hostel administration, covering students, rooms and beds, leave requests, fees, visitors and presence tracking, with secure access for students, parents or guardians, wardens, security staff, administrators and service staff.
- To implement a secure exceptional-movement workflow in which a staff-approved request produces a temporary QR token that security personnel scan and the backend verifies server-side, with every outcome logged. Routine working-day movement requires no QR.
- To provide constraint-based, explainable room allocation, together with booking of Wi-Fi and gym or recreation slots, subscription plans and payment-deadline reminders.
- To classify complaints automatically and route them by priority, and to provide a guided pre-complaint self-check (for example, MCB or valve status) before a formal complaint is registered.
- To predict maintenance needs from historical records and to maintain a digital blueprint of fire alarms, extinguishers and first-aid stations.
- To forecast meal demand and inventory requirements in order to reduce food waste and avoid stock shortages.
- To support emergency (SOS) reporting with alerts, acknowledgement tracking and a complete audit trail.
- To provide a hostel account manager for income and expenditure tracking, and an entertainment and fitness corner as a bookable hostel resource.
- To offer role-based dashboards, risk analysis and simulation for administrative planning, and a link on the college website that showcases hostel services.

1.4 SCOPE OF THE PROJECT

The scope of HostelOS covers the full operational cycle of a hostel, organized into four functional areas.

Administration: student and accommodation records, role-based authentication and access control, room and bed management, leave requests, fees and payments, visitor records, mess operations, inventory, and hostel income and expenditure.

Safety: presence tracking, authorization of exceptional movement (late returns, holiday outings, medical or personal visits) through server-verified temporary QR tokens, SOS and emergency alerts with acknowledgement and audit logging, and a digital map of fire and first-aid equipment.

Intelligence: supervised text classification for complaints, time-series and regression models for meal and inventory forecasting, risk classification for predictive maintenance, constraint optimization for explainable room allocation, and risk analysis.

Insight: role-based dashboards, analytics and simulation to support planning by wardens and administrators.

The system is designed as a layered architecture consisting of web, mobile and college-website interfaces, an application and service layer, an AI and analytics layer, a central database, and a notification and security layer. It is intended to run on a standard application server with everyday smartphones and computers as client devices; the device camera is used for QR scanning, so no dedicated RFID, NFC or biometric hardware is needed.

The project does not include the design of physical access-control hardware, and QR verification is not applied to routine daily movement. CCTV integration and Wi-Fi access-point integration are treated as optional components. The AI models are intended to support decisions made by hostel staff and not to replace them.

1.5 SIGNIFICANCE OF THE PROJECT

The significance of HostelOS lies in bringing administration, safety and intelligence together on a single platform, which the reviewed literature has not done for hostels.

For administrators and wardens, a shared data layer and role-based dashboards give a consolidated view of occupancy, complaints, finances, stock and incidents. Predictive maintenance and forecasting let them plan ahead and act before a failure or shortage occurs.

For students, automatic complaint classification and priority routing should shorten resolution time, and the pre-complaint self-check helps resolve simple faults quickly.

Explainable room allocation makes the process more transparent and easier to trust. Students also gain access to amenities and a reliable SOS channel.

For parents and guardians, the platform offers clearer visibility into leave and exceptional movement, backed by a traceable record.

For safety and security staff, limiting QR use to exceptional movement keeps the gate practical, because routine college travel is not interrupted, while unusual movement remains explicitly authorized, server-verified and logged. Invalid, expired or reused tokens are rejected and recorded. The SOS module with acknowledgement tracking and audit logging strengthens accountability in emergencies.

For hostel resources, meal-demand and inventory forecasting can reduce food waste and over-stocking, and the removal of dedicated access hardware lowers deployment and maintenance cost.

At the institutional level, HostelOS shows how a hostel can move from reactive, paper-based or fragmented digital management to a data-driven, proactive model. Because the platform records movement, complaints, payments, maintenance and incidents in one central database with audit logging, it improves transparency and accountability, and it gives institutions reliable evidence for policy and planning decisions such as resource allocation, budgeting and safety improvements. Its modular architecture and use of a standard open-source technology stack also make it affordable to adopt and easy to extend, so it can serve other residential facilities such as staff quarters or campus guest houses. From an academic perspective, the project adds to existing research by combining administration, safety and applied machine learning in one practical system, and it provides a base for future work on intelligent campus and residential management.

1.6 CONTRIBUTIONS

The project is expected to make the following contributions:

- A unified hostel platform that integrates administration, safety, predictive analytics and resource optimization, replacing the fragmented, single-purpose modules found in existing systems.
- A hardware-light exceptional-movement workflow that uses server-issued temporary QR tokens only for non-routine movement, with server-side

verification and complete logging, removing the need for RFID, NFC or biometric readers.

- An AI-assisted decision-support layer comprising complaint classification and priority routing, predictive maintenance, meal and inventory forecasting, and explainable constraint-based room allocation.
- An emergency management module with SOS reporting, alerting, acknowledgement tracking and audit logging, supported by a digital safety map.
- Role-based dashboards, analytics and simulation that turn operational data into planning information for administrators.
- A modular, practical architecture based on a standard open-source stack that can serve as a reference for similar institutional residential facilities.

1.7 ORGANIZATION OF THE REPORT

The remainder of this report is organized as follows.

Chapter 1 introduces the background, problem statement, objectives, scope, significance and expected contributions of the project.

Chapter 2 presents the literature review, covering existing work on hostel management systems, QR, RFID and biometric access control, IoT-based monitoring, and machine-learning approaches to complaint classification and forecasting. It also identifies the research gap.

Chapter 3 describes the system design and methodology, including the layered architecture, the exceptional-movement QR workflow, the database design, and the AI and analytics methods used.

Chapter 4 covers implementation, including the hardware and software specifications, the technology stack and the module-wise development carried out by each team member.

CHAPTER 2

LITERATURE REVIEW

2.1 INTRODUCTION

Hostel management has traditionally been an administrative activity centred on registers, forms and manual verification. Over the past decade, research and student projects have gradually moved from paper-based records to web-based hostel management systems, and then to sensor-based 'smart hostel' solutions that use RFID, biometrics, QR codes and Internet of Things (IoT) devices. In parallel, machine learning and natural language processing have matured in other service domains, where they are used to classify complaints, forecast demand and support decisions. However, these two lines of work have rarely been brought together in a single hostel platform.

The literature reviewed for HostelOS covers several important areas: web-based hostel administration, RFID and biometric access control, QR code-based access control and smart locks, IoT-based attendance, energy and safety monitoring, AI-based visitor detection, machine-learning classification of complaints, and a systematic review of QR based access-control architectures. In total, ten studies published between 2018 and 2026 are examined. Together, they provide the technical foundation for HostelOS and show where existing work stops short of a unified, intelligent platform that combines administration, safety, predictive analytics and resource optimization.

2.2 HOSTEL MANAGEMENT AND INTELLIGENT CAMPUS SYSTEMS

A hostel is a dense operating environment. Attendance, leave, gate movement, room allocation, fees, complaints, mess operations, inventory, maintenance, visitors and emergencies must all be handled at the same time, often by a small administrative

team. Conventional hostel management software mainly digitizes the forms that wardens and administrators already use. Its modules for fees, complaints, mess and rooms are usually built in isolation, so the data they generate is rarely used for prediction, automation or decision support.

Access control has followed a similar path. Manual registers were followed by RFID cards and biometric verification, and more recently by QR codes read through ordinary smartphone cameras. Each approach has trade-offs. RFID and biometric systems provide strong identification but need dedicated readers at every access point, which increases cost and maintenance effort. QR-based systems avoid specialized hardware, but if every routine entry and exit requires a scan, the process adds daily friction, and the security of the scheme depends heavily on how credentials are generated, validated and expired on the server.

Finally, machine learning offers techniques that suit hostel data naturally: supervised text classification for complaints, time-series and regression models for meal and inventory demand, and risk classification for maintenance. These techniques are well established in other domains, but their application to complete hostel operations has received little attention in the reviewed literature. The studies below are discussed in reverse chronological order.

2.3 REVIEW OF RELATED RESEARCH

2.3.1 QR Code-based Access Control Systems

Alsawi and Ibrahim presented a systematic literature review of QR code-based access control systems, published in the International Journal of Advanced Computer Science and Applications. The review covers studies from 2015 to 2023 drawn from major databases and organizes them into an architectural taxonomy. It also examines the security landscape of these systems and outlines future research directions. A recurring finding is that QR-based access control is attractive because it needs no specialized credential hardware, but that its security and robustness depend on design choices such as how credentials are issued and verified.

This study is relevant to HostelOS because the platform's exceptional-movement workflow is built on QR credentials. It supports the design decision that all QR validation should happen on the server, with temporary tokens that expire and are rejected and logged when reused.The limitation is that the work is survey-level only.

It identifies gaps in security and robustness but does not describe a deployed system, and it does not address how QR credentials should be tied to a student's leave or presence status in a hostel.

2.3.2 Smart Hostel Management Through the IoT

This conference paper, titled 'Smart Hostel Management Through the IoT: Reducing Energy Consumption, Securing Access, and Preventing Theft', proposes an IoT sensor network for hostels. The system uses occupancy sensing, RFID/NFC-based locks and gas leak detectors to automate energy saving and safety monitoring. It confirms that energy waste, unauthorized access and safety hazards are practical concerns in hostel environments and that automation can help address them.

The work is useful to HostelOS as evidence of the safety needs a hostel platform must serve. Its approach, however, is purely IoT- and hardware-centric. It depends on installed sensors and locks, lacks predictive analytics, and does not offer a unified administrative dashboard. HostelOS addresses safety through software, using presence tracking, SOS reporting and a digital safety map, and treats physical devices as optional.

2.3.3 Smart lock system using QR code technology

Adedoyin and Olukoya developed a smart lock system using QR code technology, published in the ABUAD Journal of Engineering Research and Development. A microcontroller-driven lock verifies a server-issued QR credential before it actuates a solenoid lock. The design shows that QR credentials issued by a server can replace physical keys and cards for door access.

The study supports the server-issued, verify-before-grant principle used in HostelOS. Its limitation is scope: it is a hardware access-control unit and is not integrated with attendance, complaints or hostel resources. A lock that only checks a credential has no knowledge of whether a student is on approved leave, so it cannot support the wider workflow that HostelOS manages.

2.3.4 Digitizes Hostel Administrative workflows

Tan's final year project at Tunku Abdul Rahman University of Management and Technology describes a full-stack web application that digitizes hostel administrative workflows and replaces manual, paper-based processes. It is a representative example of the conventional hostel management system that many institutions deploy, and it demonstrates the benefit of centralizing hostel records.

This work forms the administrative baseline against which HostelOS is positioned. The system digitizes administration only. It does not include predictive analytics or AI-based decision support, and it does not extend into safety or resource optimization. HostelOS retains the administrative core but adds an intelligence layer on top of it.

2.3.5 Smart door access control system based on QR codes

Jain and co-authors proposed a smart door access control system based on QR codes, published in the International Journal of Informatics and Communication Technology. An ESP-based door controller decodes server-issued QR codes and grants or denies physical access accordingly. The paper reinforces that low-cost microcontrollers and ordinary QR codes can provide workable access control.

Like the previous access-control studies, it concentrates on the door itself. There is no student leave or presence state behind QR issuance, so the decision to grant access is not linked to any hostel record. HostelOS differs by issuing QR credentials only after staff approval of an exceptional movement, which keeps the credential connected to an auditable request.

2.3.6 Hostel management system using artificial intelligence and IoT

Bhardwaj and co-authors proposed a hybrid smart hostel management system using artificial intelligence and IoT. Facial-recognition cameras feed an edge and cloud pipeline that performs real-time visitor detection and raises alerts. This is one of the few reviewed works that brings AI into hostel security.

The approach is camera- and edge-hardware heavy and raises privacy concerns, since it continuously processes facial data. It is also not linked to the broader administration of the hostel. HostelOS avoids dependence on facial recognition for routine operation and keeps CCTV only as an optional, controlled component.

2.3.7 IoT-based smart hostel management system

Singh and co-authors presented an IoT-based smart hostel management system at the 4th International Conference on Intelligent Computing and Control Systems (ICICCS). RFID and PIR motion sensors are networked through IoT to log attendance automatically and to control room power. The study demonstrates that attendance recording and energy control can be automated without manual registers.

Its scope is narrow, being limited to attendance and power. It does not cover complaints, mess, inventory or safety integration, and it relies on sensors deployed in the building. For HostelOS, the study confirms the value of automated presence

tracking, which the platform achieves through server-side records and exceptional-movement authorization rather than a sensor network.

2.3.8 Hostel management system with QR codes

Patel and co-authors described a hostel management system with QR codes, published in the International Journal of Computer Applications. QR codes are generated and scanned at the hostel gate for daily attendance and access verification. It is an early example of using QR codes in a hostel context.

The weakness is that QR is used for routine daily attendance, which adds friction for every student on every entry and exit. The design also lacks an exception-based or server-side authorization model. HostelOS draws a clear lesson from this: QR should be reserved for exceptional movement such as late returns, holiday outings, and medical or personal visits, while normal working-day movement needs no QR.

In contrast to the daily-attendance approach of Patel et al., HostelOS issues a QR token only after a leave or exceptional-movement request has been approved by the warden. Each token is temporary, linked to that specific request, and validated entirely on the server, so expired, invalid or reused tokens are rejected and logged. This keeps the QR mechanism secure and traceable and removes the daily scanning burden for routine movement. It also makes the scan result part of the student's leave and presence record, which the earlier QR-based hostel system did not provide.

CHAPTER 3

SYNTAX ANALYSIS AND DESIGN

3.1 INTRODUCTION

System analysis converts the problem identified in Chapter 1 into functional requirements and architectural components. HostelOS is designed as a unified, layered platform consisting of role-based user portals, a backend service layer, a central database, an AI and analytics layer, and a notification and security layer. The central design principle is that the backend is the single source of truth: every decision about authorization, gate movement and sensitive data is made on the server, while the user-facing portals only present information and collect input.

The platform integrates student accommodation, room allocation, leave, presence, gate authorization, fees, complaints, mess, maintenance, inventory, emergency response, analytics and decision support. In addition to these core areas, the design includes resident-oriented services such as Wi-Fi, gym and fitness-corner booking with subscription plans and payment deadlines, a digital fire-safety and first-aid map, a guided pre-complaint self-check, a hostel account manager for income and expenditure, and a public showcase link on the college website.

3.2 EXISTING SYSTEM

Existing hostel-management mechanisms generally fall into three categories: manual registers and spreadsheets, stand-alone hostel-management software, and hardware-based access-control systems that use RFID cards, biometric readers or routine QR scanning.

Manual registers and spreadsheets are simple to start but are difficult to search, easy to lose and cannot produce timely summaries. Stand-alone software generally digitizes forms for fees, rooms or complaints, but each function usually operates as an

isolated module with little shared data and almost no prediction or decision support. Hardware-based access control records movement but requires dedicated readers and cards at every gate, and routine QR or card scanning for every working-day entry and exit adds daily congestion without adding useful authorization.

The limitations can be summarized as follows:
The existing hostel management systems have several limitations that affect their efficiency and reliability. Although administrative activities are digitized, different modules often operate independently without proper integration. The stored data is rarely used for classification, prediction, or optimization. Complaints are handled manually without automatic categorization or priority assignment, while maintenance activities are mainly reactive, making it difficult to identify recurring problems from historical records. Meal preparation and inventory ordering are not properly forecast, which can lead to food waste and shortages. Gate control often depends on dedicated hardware or applies the same procedures to both routine and exceptional student movement, causing unnecessary delays. Emergency handling also lacks proper acknowledgement tracking and an audit trail. Furthermore, the absence of a centralized dashboard makes it difficult to monitor important information such as room occupancy, fees, complaints, student presence, maintenance, and inventory in one place. These limitations highlight the need for an integrated and intelligent hostel management system such as HostelOS.

Conventional hostel management systems face several challenges in managing daily operations effectively. Although many systems provide digital facilities for student registration, room allocation, fee collection, and complaint management, these functions are often maintained as separate modules. This lack of integration makes it difficult for administrators to obtain a complete view of hostel activities and coordinate different departments. In addition, the limited use of historical data reduces the ability of existing systems to identify patterns, predict future requirements, and support informed decision-making.

Another major limitation is the manual handling of complaints and maintenance requests. Complaints may not be automatically categorized according to their type or urgency, which can delay the resolution of important issues. Similarly, maintenance is usually performed after a problem occurs rather than being predicted in advance.

Without analyzing previous maintenance records, recurring faults may remain unnoticed, resulting in repeated repairs, increased costs, and inconvenience to residents.

3.3 PROPOSED SYSTEM

HostelOS proposes a unified intelligent platform that combines hostel administration, safety workflows and an AI-assisted decision-support layer. All modules share one database and one role-based access model, so that data entered in one module (for example, a complaint) can be used by another (for example, predictive maintenance and risk analysis).

A deliberate design decision concerns the gate. NFC/RFID has been removed from the current design. Normal working-day movement to and from college does not require any QR. A temporary, server-verified QR token is issued only when special authorization is required, such as a late return, a holiday outing, or a medical or personal visit. The proposed flow of control is:



For the QR feature the backend performs all validation; a QR image by itself is never trusted, and it carries only a secure token reference rather than student data.

3.4 FUNCTIONALITIES

The functionality of HostelOS focuses on simplifying and improving hostel management through an integrated digital platform. It provides facilities for student registration, profile management, room and bed allocation, attendance tracking, leave requests, and visitor management. Students can use the system to submit complaints, request maintenance services, book amenities, and receive important updates. The service module helps staff manage mess operations, inventory, and maintenance activities efficiently, while the finance module supports hostel fee management, payment tracking, income and expenditure records, and payment reminders.

HostelOS also incorporates intelligent features to improve decision-making and safety. AI-based complaint classification helps categorize complaints and assign priorities, while predictive maintenance identifies possible maintenance needs using historical records. Meal and inventory forecasting helps estimate demand and reduce food waste. The system includes emergency SOS alerts, fire safety information, and QR-code verification for exceptional student movement. Role-based access control ensures that students, wardens, security staff, and administrators can access the appropriate features. Overall, HostelOS reduces paperwork, improves communication, supports better resource utilization, and provides a more secure, organized, and efficient hostel management experience. The platform is designed to be user-friendly and scalable, allowing additional features to be introduced as hostel requirements grow. These functionalities make HostelOS a practical solution for improving operational efficiency, student satisfaction, and overall hostel management.

3.5 NON-FUNCTIONAL REQUIREMENTS

Usability: Each role should see only the screens and actions relevant to it. Routine tasks such as applying for leave, raising a complaint or booking a slot should require few steps, and the interface should be usable on both a desktop browser and a smartphone.

Reliability: The system should tolerate temporary network problems. A controlled fallback and synchronization approach is needed for gate verification so that movement events are not lost when connectivity is interrupted, and so that offline-recorded events can be reconciled with the server afterwards.

Security: Access to every operation must be authenticated and authorized. Passwords must be stored using secure hashing, API endpoints must enforce authorization, QR tokens must be unique, short-lived and verified on the server, and input must be validated.

Privacy: Role-based access limits who can see personal, medical, financial and movement information. Gate events indicate movement and presence state; they are not continuous location tracking. Audit logs record access to sensitive data.

Performance and Maintainability: Common operations such as dashboards, complaint registration and QR verification should respond quickly. The modular design should

allow individual modules, in particular the AI models, to be improved or replaced without changing the rest of the system.

3.6 SYSTEM ARCHITECTURE

System architecture is the overall structure of a system that explains how its different components are organized and connected to work together. In the HostelOS project, the system architecture consists of five main layers: the presentation layer, application or service layer, intelligence layer, data layer, and notification and security layer. The presentation layer allows students and staff to access the system through a web portal, mobile app, or college website. The application layer manages authentication, room allocation, leave requests, complaints, mess, inventory, payments, and finance. The intelligence layer uses AI and analytics for complaint classification, predictive maintenance, meal and inventory forecasting, room allocation, and risk analysis. The data layer stores all essential information in a central database, including student details, rooms, finances, complaints, inventory, and presence records. Finally, the notification and security layer manages emergency SOS alerts, fire safety notifications, QR verification, push notifications, and emails. These layers work together to provide a unified, intelligent, secure, and efficient hostel management system.

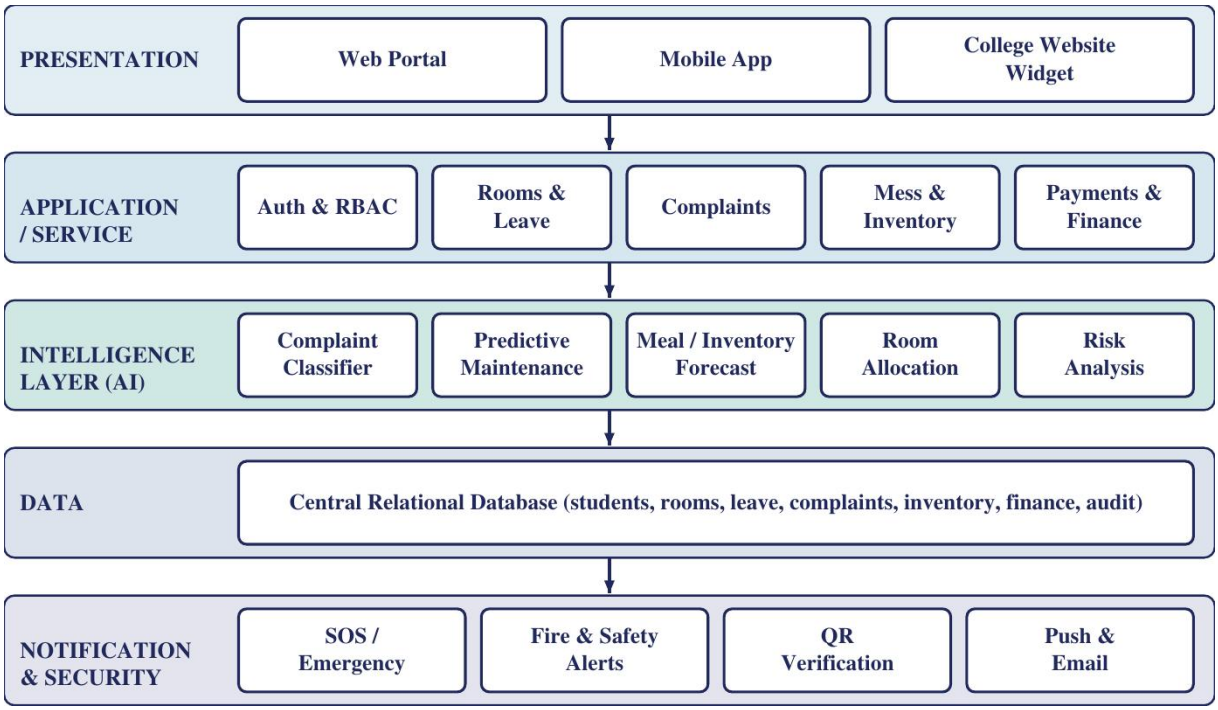


Figure 3.1: High-Level HostelOS System Architecture

The backend is the source of truth for every layer. Because all QR validation is performed on the server, the presentation layer cannot bypass authorization.

3.7 USER ROLES AND ACCESS MODEL

HostelOS defines seven logical roles, and each role can access only the information and operations needed for its duties. Students manage their own profile and room details, raise leave and exceptional-movement requests, register complaints, view fees, make bookings and trigger SOS alerts. Parents or guardians see only limited information about the associated student, such as leave status and notifications. Wardens approve leave and exceptional movement, handle complaints and incidents, and view presence and block-level data. Security personnel scan and verify QR tokens at the gate, view the movement results and acknowledge incidents. Administrators manage users, rooms, fees and finance, and have access to analytics, risk indicators and audit logs. Mess staff handle meal plans, bookings, preparation figures, waste records and kitchen inventory, while maintenance staff receive classified complaints and maintenance tasks and update their status. This role-based separation protects sensitive data, limits the effect of misuse or error, and keeps responsibilities clearly divided across the platform.

Role	Principal access and responsibilities
Student	Own profile and room, leave and exceptional-movement requests, complaints, fees, bookings, SOS.
Parent / Guardian	Limited information about the associated student, such as leave status and notifications.
Warden	Approves leave and exceptional movement, handles complaints and incidents, views presence and block-level data.
Security	Scans and verifies QR tokens at the gate, views movement results and acknowledges incidents.
Administrator	Manages users, rooms, fees and finance; views analytics, risk indicators and audit logs.
Mess staff	Manages meal plans, bookings, preparation figures, waste records and kitchen inventory.
Maintenance staff	Receives classified complaints and maintenance tasks and updates their status.

Table 3.1: Roles and Access Scope

Authentication: Each user signs in with credentials that are verified against securely hashed passwords. A successful sign-in produces a session or token that identifies the user and the role.

Authorization: Every API request is checked against the role before data is returned or an action is performed. Authorization is applied on the server regardless of what the user interface displays.

3.8 GATE AUTHORIZATION MODEL

Normal Movement: Normal working-day college movement during permitted hours does not require a QR. Hostel presence is maintained through the normal hostel presence process and the approved leave state, and the system shows states such as IN, OUT or ON LEAVE. Academic college attendance is a separate matter and is not handled by this QR feature.

Exceptional Movement: A QR token is required only when special authorization is needed, for example when a student returns after the permitted hostel time, leaves on a holiday or closed day, makes a medical visit, or goes on a personal outing. This keeps the gate free of routine congestion while ensuring that unusual movement is explicitly authorized and traceable.

QR Token Design: The QR code contains only a secure token or reference. The details of the permission are held on the server. A token is therefore a pointer, not a document, and its value is limited if it is copied. A conceptual token record is:

$$T = \{tokenID, studentID, requestID, purpose, validFrom, expiry, status\}$$

At scan time the backend accepts the movement only if the token exists, has not expired, has not been used, belongs to the associated student, matches an approved permission and is compatible with the student's current presence state:

$$Valid(T) = exists(T) \text{ AND } now < expiry \text{ AND } status = UNUSED \text{ AND student matches AND permission approved AND state compatible}$$

3.9 USER WORKFLOW EXAMPLES

Holiday Outing: The student first submits an exceptional movement request through the HostelOS portal. The warden or an authorized staff member reviews the request and approves it. Once approved, the backend generates a unique, temporary QR token

and displays it to the student. At the hostel gate, the student presents the QR code, which is scanned by the security officer. The backend verifies the token's validity, expiry time, student association, and permission before allowing the movement.

After successful verification, the movement is recorded, the student's presence status is updated to OUT, and the activity is stored in the audit log. When the student returns, additional authorization is required if the return occurs after the permitted time; otherwise, normal presence rules apply. Any invalid, expired, or previously used QR token is rejected and recorded in the system log. This workflow improves hostel security, maintains accurate student movement records, and ensures that exceptional outings are properly authorized and monitored.

Complaint with Pre-Complaint Self-Check: The student starts a complaint through the HostelOS system and selects the appropriate category. Before the complaint is finally registered, the system provides a guided self-check to identify simple problems, such as a closed water valve or a tripped miniature circuit breaker (MCB). If the student resolves the issue through this self-check, a formal complaint is not registered, helping reduce unnecessary complaints.

If the problem remains unresolved, the system registers the complaint, automatically classifies it, and forwards it to the maintenance staff according to its priority. This helps ensure that urgent issues receive appropriate attention and are handled efficiently. The complaint details and resolution status are stored in the database for future reference. These records can later be used for predictive maintenance and risk analysis, helping administrators identify recurring problems and improve hostel maintenance services.

Room Allocation Workflow: When a new student registers in HostelOS, the administrator enters or verifies the student's details and room requirements. The system checks room availability, capacity, and allocation constraints before suggesting a suitable room or bed. Once the allocation is approved, the student's room details are stored in the database and updated in the administrative dashboard. This process reduces manual work, avoids duplicate allocations, and improves room management.

Emergency SOS Workflow: When a student faces an emergency, they can activate the SOS feature through the HostelOS platform. The system records the emergency request and sends an alert to the authorized hostel staff. The responsible staff member acknowledges the alert and takes the necessary action. The system tracks the

acknowledgement and records the response details for future reference. This workflow helps improve emergency coordination, accountability, and student safety.

Predictive Maintenance Workflow: HostelOS stores historical maintenance records, including previous complaints, repair activities, and recurring faults. The system analyses these records to identify patterns that may indicate future maintenance requirements. When a potential issue is identified, the system can alert administrators or maintenance staff to inspect the affected facility. This workflow supports early problem detection, reduces repeated faults, and helps minimize maintenance costs.

These examples show that the platform handles both safety-related workflows and everyday service workflows in a single role-controlled system.

3.10 SECURITY AND PRIVACY DESIGN

Security is integrated into the architecture rather than treated as a separate add-on.

Authentication and Password Protection: Users are authenticated before any protected operation, and passwords are stored only as secure hashes.

Role-Based Access Control: RBAC separates permissions according to role. A student, warden, security officer and administrator do not have identical access to system resources.

Temporary, Server-Verified QR Tokens: QR tokens are unique and short-lived, and are verified by the backend. A token that is expired, already used or not associated with the scanning context is rejected.

Input Validation and API Authorization: All inputs are validated, and every API endpoint checks the caller's role before returning data or applying a change.

Audit Logging: Sensitive operations, including approvals, gate scans, SOS handling and access to protected records, are recorded with the actor, action, resource and timestamp. Audit records support accountability and investigation.

Controlled CCTV: CCTV is treated as an optional, controlled security component with defined access, retention and audit policies. It complements, rather than replaces, the gate and incident workflow.

Privacy and Access Control: The system ensures that students, wardens, security staff, and administrators can access only the information relevant to their roles, helping prevent unauthorized access and maintain privacy.

Data Backup and Recovery: Regular backups should be maintained to protect

important records and support recovery after accidental deletion or system failure.

Data Encryption: Sensitive student information and financial records are encrypted both in transit and at rest to prevent unauthorized access. All communication between the web portal, mobile app and backend takes place over secure HTTPS (TLS) connections, so data cannot be read or altered while it travels across the network. Stored data, including personal details, payment records and credentials, is protected through encrypted database storage, and passwords are saved only as salted hashes rather than in plain text. Together with JWT-based authentication and role-based access control, this ensures that even if storage or network traffic is intercepted, the information remains unreadable to unauthorized parties.

3.11 UML-ORIENTED SYSTEM VIEWS

3.11.1 Use Case View

The principal actors are the student, parent or guardian, warden, security, administrator, and mess and maintenance staff. The student initiates leave, movement, complaint, booking and SOS requests. The warden and administrator approve and oversee. Security verifies QR tokens at the gate.

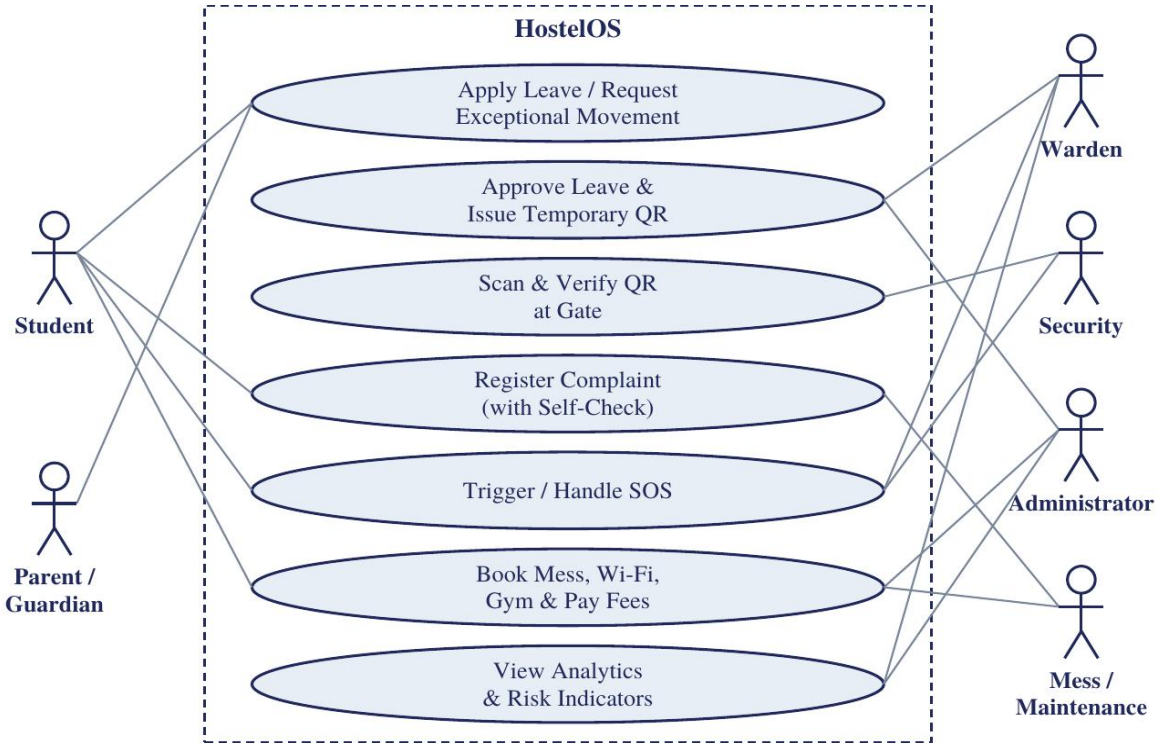


Figure 3.2: Simplified Use Case View

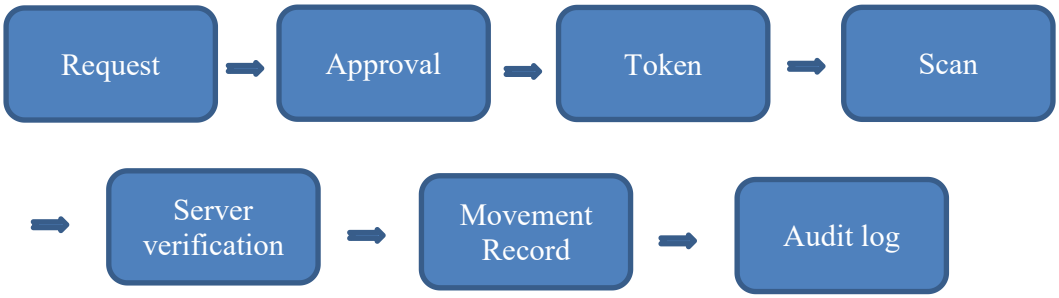
The use case diagram of HostelOS represents the interaction between different users and the hostel management system. It shows the main activities performed by students, parents or guardians, wardens, security staff, administrators, and mess or maintenance staff. The rectangle represents the system boundary of HostelOS, while the oval shapes represent the functions or services provided by the system. The lines connecting the users to the ovals show which functions they interact with.
Figure 3.2 is a simplified UML use case diagram. It shows what each type of user can do in HostelOS. The dashed rectangle is the system boundary, each oval inside it is a function the system provides, and each line connects an actor (a user role) to a function it takes part in. Students are on the left, along with parents or guardians. Wardens, security, administrators and mess or maintenance staff are on the right.

The student connects to the functions they use directly: applying for leave or requesting exceptional movement, registering a complaint with the guided self-check, triggering an SOS, and booking mess, Wi-Fi or gym slots and paying fees. The parent or guardian connects only to the leave and movement function, which reflects their limited, view-oriented access. On the other side, the warden approves leave and issues the temporary QR token, and also takes part in handling SOS incidents. Security personnel scan and verify the QR at the gate and acknowledge SOS incidents. The administrator is linked to approvals, bookings and fees, and analytics with risk indicators. Mess and maintenance staff are linked to the complaint function, where they receive classified complaints and tasks, and to the booking function, where they manage meal plans and bookings.

The diagram matches the role-based access model in Section 3.7, because each actor reaches only the functions needed for its duties. It also shows the exceptional-movement flow across three roles: the student requests, the warden approves and issues the token, and security verifies it.

3.11.2 Sequence View

The sequence for exceptional movement begins with a student's request and ends with a server-side record and audit entry:



The figure shows the exceptional-movement sequence as seven connected steps, read left to right across the first row and then continuing on the second row. The process starts with a Request raised by the student for a late return, holiday outing, or medical or personal visit. The warden or authorized staff gives Approval, after which the backend generates a temporary Token in the form of a QR code. At the gate, security personnel perform the Scan with a device camera, and the scanned data is sent to the backend for Server Verification, where the token is checked for validity.

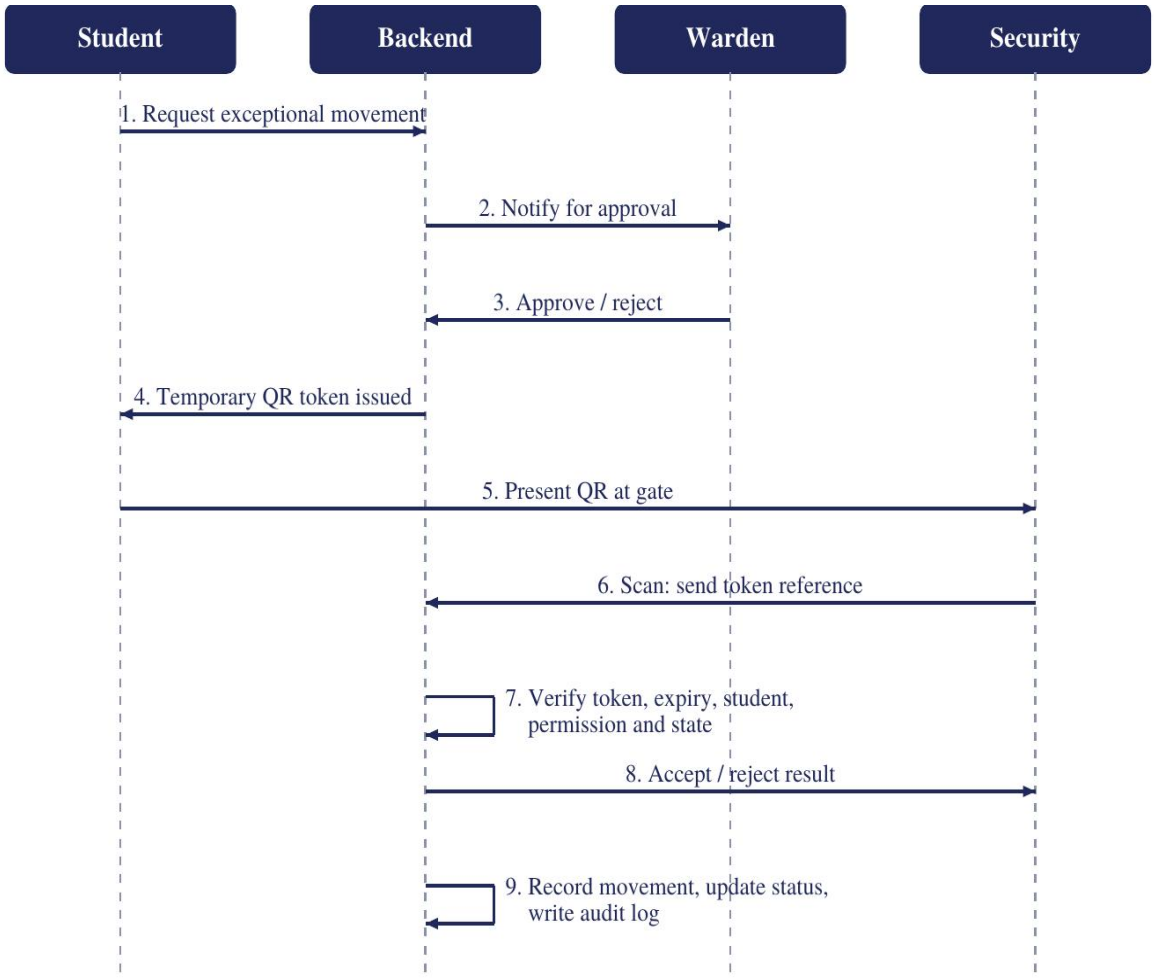


Figure 3.3: Sequence for Exceptional-Movement QR Authorization

Figure 3.3 is a UML-style sequence diagram showing how an exceptional-movement QR authorization is exchanged between four participants: the Student, the Backend, the Warden and Security. Each vertical dashed line is a participant's lifeline, and the numbered horizontal arrows show the messages in time order from top to bottom.

The process begins when the student sends a request for exceptional movement to the backend (step 1). The backend notifies the warden for approval (step 2), and the warden approves or rejects the request (step 3). On approval, the backend issues a temporary QR token to the student (step 4). When the student reaches the gate, they present the QR code to the security personnel (step 5), who scan it and send the token reference to the backend (step 6). The backend then verifies the token, its expiry, the student, the permission and the current state (step 7), and returns an accept or reject result to security (step 8). Finally, the backend records the movement, updates the student's status and writes an audit log entry (step 9).

Steps 7 and 9 are drawn as self-pointing arrows because the backend performs them internally, with no other participant involved. The diagram also shows that the backend is the central authority. The student, warden and security staff never communicate directly except for step 5, which is the physical presentation of the QR code at the gate, and every decision and record is made server-side. This keeps the process secure and traceable, because a token that is invalid, expired or already used is rejected and logged.

3.12 DATA ARCHITECTURE

The data model is relational. Its principal entities are the Student, Room and Bed, Leave Request, QR Token, Movement Record, Complaint, SOS Incident, Payment and Booking, together with supporting entities for maintenance history, meals, inventory, financial transactions, fire-safety assets and the audit log.

A student is associated with a room and bed and can create leave requests, complaints, SOS incidents, payments and bookings. An approved leave or exceptional-movement request can result in a QR token. A scanned token creates a movement record. Audit events are written for sensitive operations and refer to the acting user and the affected resource, so they are not drawn as a separate relationship in the figure.

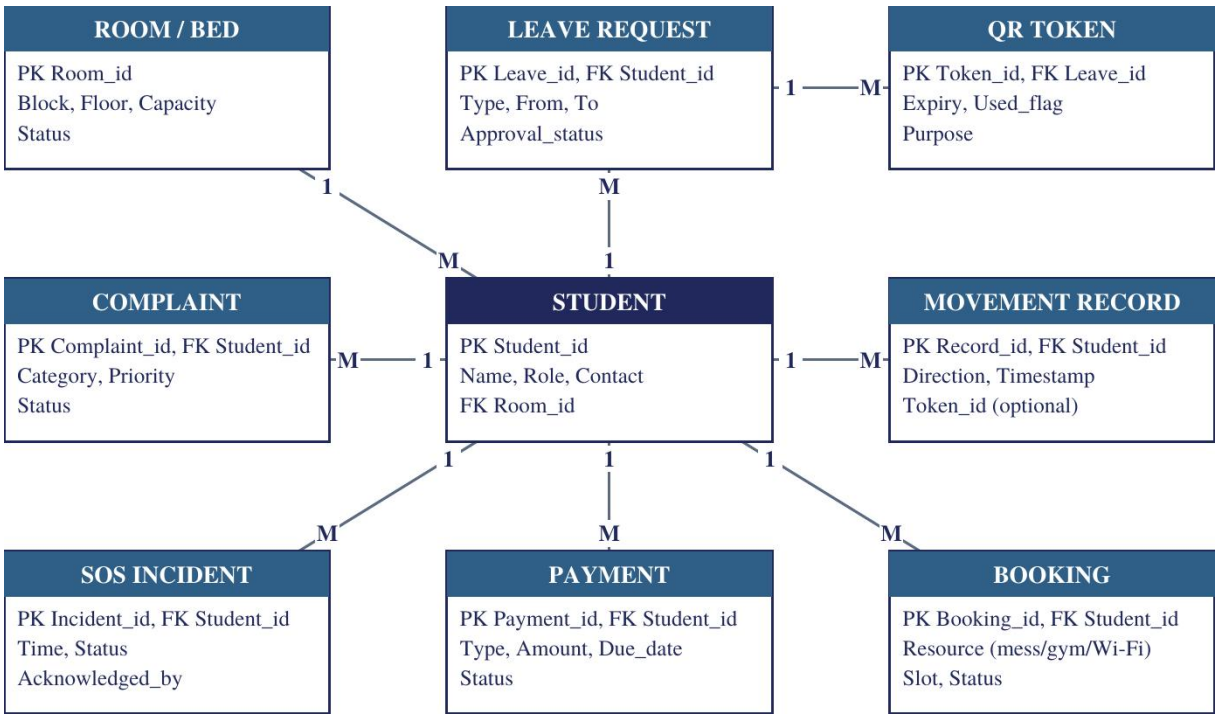


Figure 3.4: Conceptual Entity-Relationship Diagram of HostelOS

Figure 3.4 shows the conceptual Entity-Relationship (ER) diagram of HostelOS, with the STUDENT entity at the centre as the main link between the modules. Student records hold the primary key Student_id along with name, role, contact details and a foreign key Room_id. A room or bed can be assigned to many students (1:M), and each student can raise many complaints, submit many leave requests, make many payments and bookings, and be associated with many SOS incidents and movement records, so all of these entities carry Student_id as a foreign key. The exceptional-movement workflow is modelled through the LEAVE REQUEST, QR TOKEN and MOVEMENT RECORD entities. One approved leave request can generate multiple QR tokens (1:M), each with an expiry time, a used flag and a purpose, so that expired or reused tokens can be rejected. A movement record stores the direction and timestamp of each entry or exit and optionally links to the Token_id used, because routine movement needs no token. The COMPLAINT entity stores the category, priority and status that support AI-based classification and routing, SOS INCIDENT records the time, status and the person who acknowledged it for audit purposes, PAYMENT tracks type, amount, due date and status, and BOOKING covers mess, gym and Wi-Fi resource slots. Together, the diagram shows how the platform keeps all student-related operations in one structured database with clear relationships.

3.13 DATA ELEMENTS

Entity	Principal attributes	Purpose
Student	Student_id, Name, Role, Contact, Room_id	Identifies the resident and links all records.
Room / Bed	Room_id, Block, Floor, Capacity, Status	Accommodation and allocation.
Leave Request	Leave_id, Student_id, Type, From, To, Approval_status	Digital leave and authorization.
QR Token	Token_id, Leave_id, Expiry, Used_flag, Purpose	Temporary exceptional-movement authorization.
Movement Record	Record_id, Student_id, Direction, Timestamp	Gate and presence history.
Complaint	Complaint_id, Student_id, Category, Priority, Status	Complaint handling and classification.
Maintenance History	Task_id, Asset/Area, Fault type, Date, Resolution	Input to predictive maintenance.
Meal / Consumption	Meal_id, Date, Booked, Prepared, Consumed, Wasted	Meal forecasting and waste analysis.
Inventory	Item_id, Stock, Usage, Reorder_level	Inventory forecasting.
SOS Incident	Incident_id, Student_id, Time, Status, Acknowledged_by	Emergency response and audit trail.
Payment / Subscription	Payment_id, Student_id, Type, Amount, Due_date, Status	Fees, subscriptions and deadlines.
Booking	Booking_id, Student_id, Resource, Slot, Status	Mess, gym, fitness-corner and Wi-Fi booking.
Transaction	Txn_id, Category, Income/Expense, Amount, Date	Hostel account manager.
Safety Asset	Asset_id, Type, Block/Floor, Location, Last_check	Fire alarm, extinguisher and first-aid map.
Audit Log	Log_id, Actor_id, Action, Resource, Timestamp	Accountability and investigation.

Table 3.2: Major Data Entities and Attributes

3.14 THREAT CONSIDERATIONS

Because HostelOS handles personal records, finances, movement and emergency events, several threat categories must be considered.

QR Misuse: A student could try to share, copy or reuse a QR. Tokens are unique, short-lived and server-verified, are bound to a student and a permission, and are marked used after acceptance. The server rejects and logs invalid, expired or reused tokens.

Unauthorized Data Access: Without authorization checks, a user could view another

person's records. RBAC and API authorization limit data to the role that needs it, and audit logs record access to sensitive information.

Credential Compromise: Secure password hashing and authentication reduce the impact of a database leak or password guessing.

Tampering with Client Input: Because users can modify what their browser sends, the server validates all inputs and does not trust any value that arrives from the client, including a QR image.

Network Failure at the Gate: If connectivity fails, a controlled fallback and later synchronization procedure is used so that movements are not lost and are reconciled with server records.

Misleading AI Output: AI classifications and predictions can be wrong. Outputs are evaluated with standard metrics and human correction is allowed, so that automated suggestions assist staff rather than replace their judgement.

Denial-of-Service Attacks: Attackers may overload the HostelOS server with excessive requests, making the system unavailable to students and staff. Rate limiting and traffic monitoring can help maintain service availability.

Notification Spoofing: Fake messages may be sent to students pretending to be official hostel notifications. Notifications should be delivered through authenticated channels and verified system accounts.

Data Loss or Corruption: Hardware failure, software errors, or accidental deletion may damage important records. Regular backups, recovery testing, and transaction controls help maintain data integrity.

Malicious File Uploads: Users may upload harmful files through complaint or document-submission features. File-type validation, size restrictions, and malware scanning can reduce this risk.

CHAPTER 4

IMPLEMENTATION

4.1 INTRODUCTION

This chapter defines the implementation requirements and the major modules of HostelOS. The implementation combines a web and mobile-friendly front end, a backend API, a relational database, an AI and machine-learning environment, authentication with RBAC, QR generation and validation, dashboards, notifications and logging.

The project format specifies software requirements, hardware requirements and system modules as the principal parts of the implementation chapter. The technology names given here are the proposed choices for the project; the final names should match the stack that the team actually implements. The chapter describes the planned design of each module. The status of the actual implementation is reported separately and should reflect the team's real progress, and no module is described here as completed unless it has been built.

4.2 SOFTWARE REQUIREMENTS

The software architecture is divided into four computational layers.

Frontend: The user interface is delivered as a web application for administrators, wardens and staff, and as a mobile-friendly interface or mobile application for students and security personnel. The security interface needs access to the device camera to scan QR codes. A React-based web frontend and a Flutter or React Native mobile application are proposed.

Backend API: The backend exposes a REST-style HTTPS API that implements business logic for authentication, RBAC, rooms, leave, gate authorization, complaints, mess, inventory, payments and notifications. FastAPI (Python) or Express (Node.js) is proposed. Python is attractive because the AI components can then share a codebase

with the API. The backend validates each request and enforces authorization before any data is returned.

Database and Persistence: A relational database, for which PostgreSQL is proposed, stores users, rooms, leave requests, QR tokens, movement records, complaints, maintenance history, meals, inventory, payments, transactions, safety assets and audit events. A relational model suits the many one-to-many relationships between a student and that student's records.

AI and Machine-Learning Environment: The intelligence layer uses Python libraries such as scikit-learn for text classification and risk prediction, and time-series or regression methods (for example Prophet or standard regression) for meal and inventory forecasting. Models are trained on the hostel's historical data and are evaluated with the metrics described in Section 4.4.3.

Development, Testing and Evaluation Tools: Git is used for version control, Docker for reproducible deployment, and an agentic development environment (Google Antigravity) as an aid during development. Automated tests should cover authorization correctness, QR acceptance and rejection, and the accuracy of AI components, and the evaluation layer should record timestamps so that response times can be measured.

Data and Security Requirements: The system must store student information, room details, complaints, financial records, inventory information, and movement logs accurately. It should use authentication, role-based access control, secure communication, regular backups, and proper data recovery mechanisms to protect information.

User Requirements: The system should provide different access permissions for students, wardens, administrators, and security staff. Each user should be able to access only the features and information relevant to their role. The interface should be simple and easy to understand.

These requirements work together with the software requirements to make HostelOS a practical and intelligent hostel management platform. Together, these software components support the efficient, secure, and intelligent operation of the HostelOS platform.

Layer	Proposed technology	Purpose
Frontend (web)	React	Role-based portals and dashboards
Mobile	Flutter / React Native	Student and security mobile access, QR display and scanning
Backend / API	FastAPI (Python) or Express (Node.js)	Business logic, REST API, authorization
Database	PostgreSQL	Persistent relational storage
AI / ML	scikit-learn, Prophet / regression	Classification, forecasting and risk prediction
Authentication	JWT with RBAC middleware; hashed passwords	Identity and access control
QR	QR generation library with server-side token validation	Temporary exceptional-movement passes
Notifications	Push and e-mail services	Alerts, reminders and status updates
Dev and hosting	Git, Docker, cloud VM or college server	Version control and deployment

Table 4.1: Proposed Software Requirements of HostelOS

4.3 HARDWARE REQUIREMENTS

Because NFC/RFID has been removed from the design, HostelOS needs no specialized gate hardware. Its hardware requirements are those of a conventional web service together with ordinary user devices.

Component	Specification	Purpose
Application server	2 to 4 vCPU, 4 to 8 GB RAM, Linux (Ubuntu or equivalent)	Runs the backend API and AI services
Database server	Same or dedicated instance with persistent SSD storage	Stores all hostel data and audit records
Smartphones	Android or iOS with a camera	Student access; QR display; security QR scanning
PCs / laptops	Standard browser-capable computers	Administrator, warden and staff dashboards
Network	Hostel LAN, router or switch, Internet connection	Connectivity between users and the server
Wi-Fi access points	Existing hostel Wi-Fi infrastructure	Supports the Wi-Fi amenity and subscription feature
CCTV (optional)	Existing or new IP cameras	Optional controlled security component

Table 4.2: Hardware Requirements of HostelOS

Application Server: The application server hosts the API and the AI services. The stated size is sufficient for a college-scale deployment or a demonstration, and can be enlarged if the number of users grows.

Database Server and Storage: The database needs reliable persistent storage and regular backups because it holds the audit trail and financial records. Using solid-state storage improves response times for dashboards and queries.

Client Devices: Students and security staff use smartphones, whose cameras are sufficient for scanning QR codes, so no dedicated scanner is needed. Administrators and wardens use ordinary computers.

Network and Peripheral Equipment: Standard hostel networking carries all traffic. Wi-Fi access points support the Wi-Fi amenity. CCTV, if adopted, is an optional peripheral subject to access, retention and audit policies.

Absence of Dedicated Gate Hardware: Removing NFC/RFID eliminates card readers, cards and the associated maintenance, and reduces cost and operational complexity. The gate depends only on a smartphone camera and the backend.

4.4 SYSTEM MODULES

HostelOS is divided into four principal modules for implementation:

- Core Administration, Identity and Movement Authorization Module.
- Safety, Emergency and Gate Verification Module.
- Intelligence and Analytics Module.
- Resident Services and Finance Module.

The modules share one database and one authorization layer. Each can be developed and demonstrated independently, and they are integrated through the backend API.

4.4.1 Core Administration, Identity and Movement Authorization Module

This module forms the administrative foundation of HostelOS. It manages who the users are, where they live and whether they are permitted to move.

The module also aims to recommend suitable room allocations in an explainable manner, allowing administrators to understand the reasons behind allocation decisions. It manages student leave requests and approval processes while maintaining accurate

hostel presence statuses, such as IN, OUT, and ON LEAVE. These features help administrators monitor student accommodation and movement efficiently.

Another important objective is to manage exceptional movement requests by receiving and approving requests and generating temporary QR tokens for authorized students. These tokens can be verified at the hostel gate to ensure that student movement is properly authorized. Overall, the module aims to improve administrative efficiency, strengthen hostel security, maintain accurate records, and provide reliable control over student identity, accommodation, and movement

Authentication and RBAC: A user signs in, and the backend issues a token that identifies the user and the role. Each protected endpoint checks the role before it proceeds. Passwords are stored as secure hashes.

Room Allocation: Room allocation uses room capacity, availability and permitted constraints or preferences. A constraint-based optimization or an explainable scoring method ranks feasible assignments. A feasible assignment must satisfy availability and capacity, and feasible rooms can then be ranked by a weighted score:

$$Score(s, r) = w_1 f_1(s, r) + w_2 f_2(s, r) + \ldots + w_n f_n(s, r)$$

where f_i measures how well room r satisfies the i-th permitted criterion for student s and w_i is its weight. Because the score is a sum of named terms, the system can state which criteria produced a recommendation, which supports human review and correction.

Leave and Presence: A student submits a leave request that is approved or rejected by authorized staff. Presence is maintained as a state determined by recorded events and approved leave:

$$\text{IN} \leftrightarrow \text{OUT} \leftrightarrow \text{ON LEAVE}$$

Normal working-day movement does not change the process, and no QR is requested for it. Academic attendance is separate from hostel presence.

Exceptional-Movement Request and Token Generation: When a student needs special authorization, such as a late return, a holiday outing or a medical or personal visit, the student submits a request. After authorized staff approve it, the backend generates a unique temporary token with an expiry and associates it with the student and the permission. The QR encodes only the token reference.

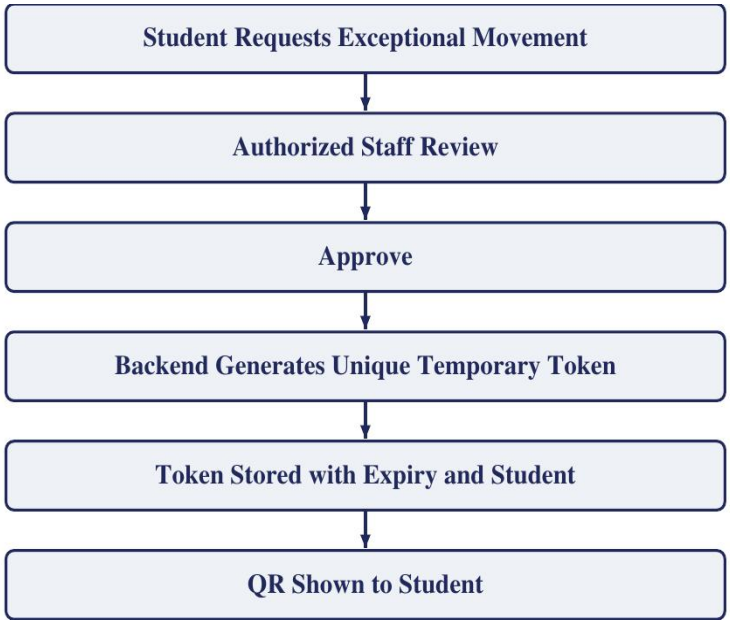


Figure 4.1: Exceptional-Movement Request and Token Generation

4.4.2 Safety, Emergency and Gate Verification Module

This module protects residents and ensures that gate decisions are made on the server. It also covers the safety-related resident features. The module maintains a digital map of important safety facilities, including fire alarms, fire extinguishers, and first-aid stations, allowing residents and authorized staff to locate them when needed. It also provides a guided self-check feature before the final registration of complaints, helping students identify and resolve simple problems. In addition, the module maintains an audit trail of security-related events to support accountability and future investigation.

Server-Side QR Verification: When security scans a QR, the application sends the token reference to the backend, which checks every condition and returns an accept or reject result. A QR image alone is never trusted. The backend accepts a scan only when the following all hold:

- the token exists and has not been tampered with;
- the token has not expired and has not already been used;
- the token belongs to the associated student;
- the underlying permission is approved; andthe movement is consistent with the student's current presence state.

- A valid scan records the movement, updates the presence state and marks the token used. An invalid, expired or reused token is rejected and logged. If the network fails, a controlled fallback and later synchronization procedure is applied.

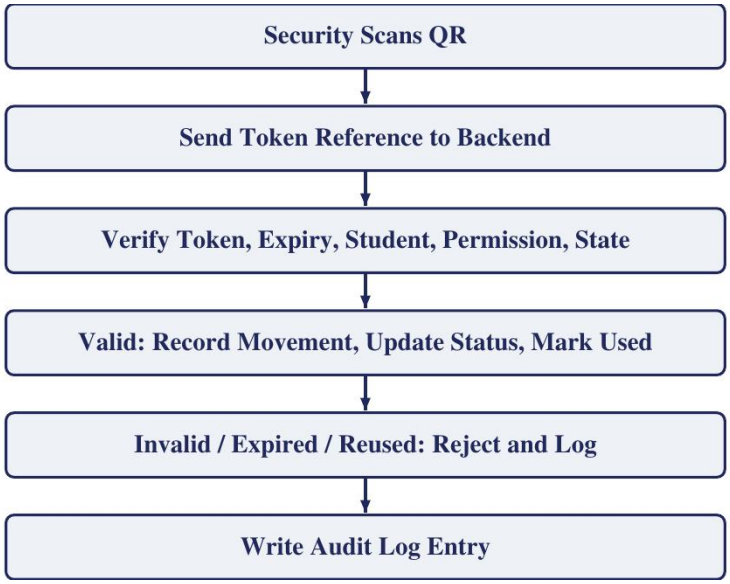


Figure 4.2: Server-Side QR Verification Workflow

Emergency and SOS: A student can trigger an SOS request. The backend records the incident, alerts authorized personnel, tracks acknowledgement and response status, and maintains an audit trail. An incident can be modelled with the states:



Fire-Safety and First-Aid Map: The digital blueprint records the location of fire alarms, extinguishers and first-aid stations by block and floor, together with the date of the last check. Staff can see which equipment is due for inspection, and in an emergency the map shows where equipment is located. Fire and safety alerts are sent through the notification layer.

Pre-Complaint Self-Check: Before a complaint is finally registered, the student is guided through a short checklist appropriate to the category, for example whether a water valve is closed or whether an MCB has tripped. If the check resolves the issue the complaint is not registered; otherwise it proceeds to registration and to the

classification described in Section 4.4.3. This reduces avoidable complaints and saves maintenance effort.

4.4.3 Intelligence and Analytics Module

This module is the feature that most distinguishes HostelOS from conventional hostel software. It converts stored operational data into classification, prediction and decision support. The methods below are proposed; the exact algorithms are to be finalized after the available data has been examined.

Another important objective is to forecast meal demand and analyse food waste using available hostel data. This helps administrators plan food preparation more effectively, reduce wastage, and ensure sufficient food availability for residents. The module also predicts inventory requirements to support the timely purchase and management of essential supplies, avoiding both shortages and excess stock.

Furthermore, the module uses risk analysis to identify areas that require attention and helps administrators make better decisions based on operational data. It presents important operational indicators, such as complaint status, maintenance risks, meal demand, and inventory requirements, through the administrator dashboard. Overall, the module aims to improve operational efficiency, support data-driven decision-making, reduce resource wastage, and enhance the quality of hostel services.

Complaint Classification: Complaints such as electrical, plumbing, cleaning, internet, furniture and other operational categories are classified by supervised text classification. The predicted category and priority determine routing. Performance is measured using accuracy, precision, recall and F1-score:

$$Precision = TP / (TP + FP),\ \ Recall = TP / (TP + FN),\ \ F1 = 2 \times Precision \times Recall / (Precision + Recall)$$

where TP, FP and FN are the numbers of true positives, false positives and false negatives for a category. Staff can correct a wrong classification, and corrected labels can be used to improve the model.

The module uses risk analysis to identify areas that require attention and helps administrators make better decisions based on operational data. It presents important operational indicators, such as complaint status, maintenance risks, meal demand, and inventory requirements, through the administrator dashboard. Overall, the module aims to improve operational efficiency, support data-driven decision-making, reduce resource wastage, and enhance the quality of hostel services.

Predictive Maintenance: Historical maintenance and complaint records are used to identify recurring infrastructure problems and estimate maintenance risk, so that preventive maintenance can be scheduled before repeated failures occur. The task is treated as a classification or risk-prediction problem and is evaluated by its prediction performance on held-out records.

Meal Forecasting and Food-Waste Analysis: Meal booking and consumption patterns are used to estimate future demand and reduce overproduction, using a time-series or regression method. Forecast error is measured with MAE, RMSE or MAPE:

$$MAE = (1/n)\ \Sigma\ |A_t - F_t|$$

$$RMSE = sqrt(\ (1/n)\ \Sigma\ (A_t - F_t)^2\ )$$

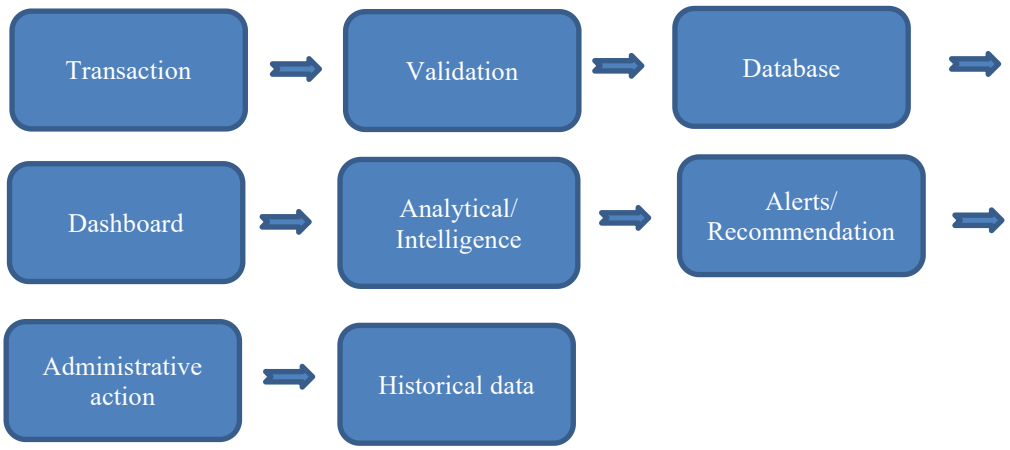
$$MAPE = (100/n)\ \Sigma\ |A_t - F_t|\ /\ |A_t|$$

where A_t is the actual consumption and F_t is the forecast value for period t. Food-waste analysis compares prepared meals, expected demand, consumption and recorded waste to find recurring patterns:

$$Waste\ rate = (Prepared - Consumed)\ /\ Prepared$$

Inventory Forecasting: Historical usage, current stock and consumption patterns estimate when and how much to reorder, and identify low-stock conditions before an item runs out.

Hostel Risk Analysis: Authorized operational indicators, such as unusual movement patterns, repeated incidents, unresolved complaints and recurring maintenance issues, are combined to identify areas that need attention. The approach can use rules or anomaly indicators, and predictive models where data permits. Risk analysis is decision support it points administrators to areas worth review and does not replace their judgement.



Administrator Analytics*:* The dashboard presents occupancy, room utilization, fee collection, presence, complaints, maintenance, meal consumption, inventory and other operational measures. The overall flow of data through the system is:

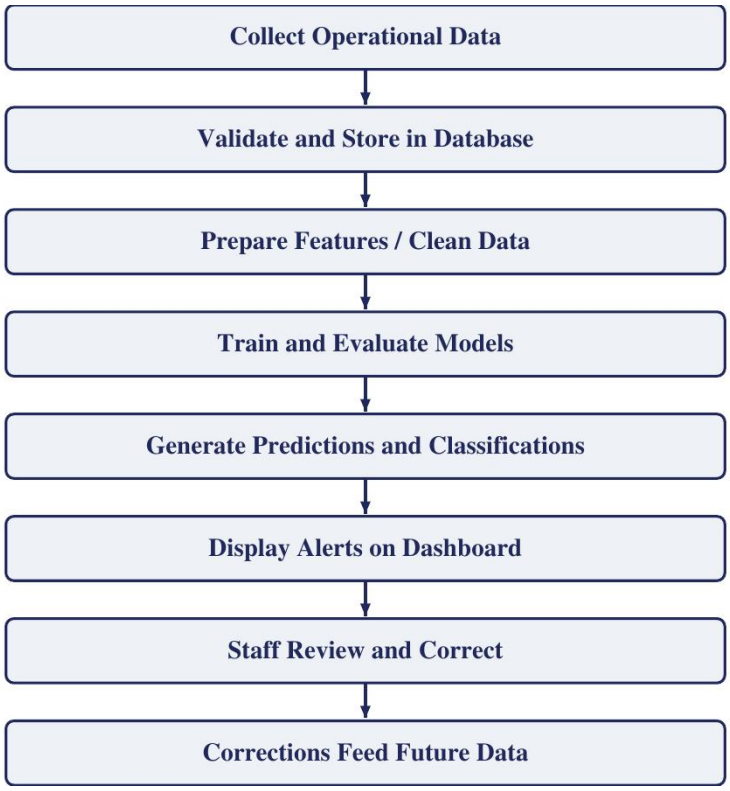


Figure 4.3: Intelligence and Analytics Workflow

Optional Advanced Components: An AI hostel assistant, a digital hostel simulation and CCTV integration are advanced or optional components, to be attempted only after the core and intelligence modules are in place. The assistant is a natural-language interface connected to authorized backend functions, and RBAC must be enforced before any sensitive information or action is exposed. The simulation models situations such as increased intake, room closures, maintenance shutdowns and accommodation shortages to support planning.

4.4.4 Resident Services and Finance Module

This module handles the everyday services that residents use and the financial administration of the hostel.

Another important objective is to maintain the hostel account manager, which records income and expenditure and supports transparent financial management. The module also facilitates the booking of services such as Wi-Fi, gym facilities, fitness corners,

and entertainment services. These features allow students to access hostel amenities conveniently and help administrators monitor their usage.

Furthermore, the module provides notifications and reminders regarding bookings, payment deadlines, and other important hostel activities. It also provides a public showcase link that can be integrated into the college website to display relevant hostel information. Overall, this module aims to improve resident services, simplify financial operations, reduce manual administrative work, and provide a convenient and organized hostel management experience.

Mess and Inventory Management: Meal bookings and consumption are recorded, and mess staff maintain kitchen inventory. These records feed the forecasting and food-waste analysis in Section 4.4.3.

Fees, Subscriptions and Payment Deadlines: Fees and subscription plans, such as Wi-Fi or gym access, are recorded with an amount and a due date. The system raises reminders as a deadline approaches and flags overdue payments. The administrator dashboard reports fee collection.

Hostel Account Manager: The account manager records income, such as fees and subscriptions, and expenditure, such as food supplies, maintenance and utilities, so that the administration has a transparent financial view. The balance for a period is:

$$\textit{Net balance} = \textit{Total income} - \textit{Total expenditure}$$

Amenity and Fitness-Corner Booking: Residents can book slots for the gym, the entertainment area and the fitness corner. Each slot has a capacity, and a booking is accepted only when capacity remains:

$$\textit{Accept booking if Booked (slot)} < \textit{Capacity (slot)}$$

Notifications: The notification service sends reminders for payment deadlines, leave and movement approvals, complaint status, safety inspections and emergency alerts, through push or e-mail messages according to role.

Public Showcase Link: A public page or widget, linked from the college website, presents the hostel's services and facilities. It exposes only non-sensitive, public information and does not provide access to any resident data.

HostelOS integrates the residents, service, and finance modules into a single platform to simplify hostel management. The resident’s module manages student information and room allocation, the service module handles complaints, maintenance, and hostel

facilities, and the finance module manages fees, payments, and financial records. Together, these modules reduce manual work, improve accuracy, increase transparency, and ensure efficient hostel operations.

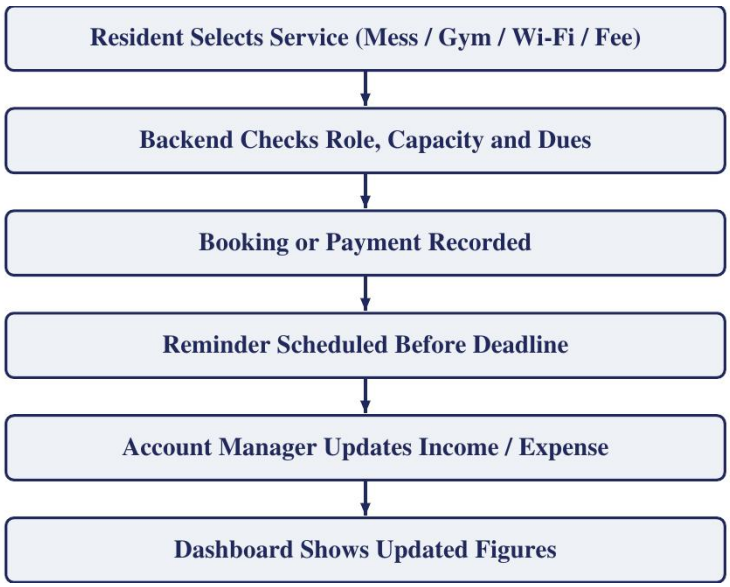


Figure 4.4: Resident Services and Finance Workflow

4.4.5 Module Allocation among Team Members

The modules are divided among the four team members so that each one's scope is self-contained and can be demonstrated independently. This is the proposed allocation; the final contribution record must reflect the work each member actually performs.

Team member	Register No.	Module
Gokul SR	BMC23CS056	4.4.1 Core Administration, Identity and Movement Authorization
Jeen George	BMC23CS057	4.4.2 Safety, Emergency and Gate Verification
S Adithyan	BMC23CS094	4.4.3 Intelligence and Analytics
Albi Thomas	LBMC23CS105	4.4.4 Resident Services and Finance

Table 4.3: Proposed Module Allocation

CHAPTER 5
CONCLUSION

5.1 CONCLUSION

This project proposed HostelOS, an intelligent and unified platform for hostel operations, safety and resource optimization. The work began from the observation that conventional hostel management software mainly digitizes administrative forms, runs as isolated modules, and makes little use of the data it collects for prediction or decision support. The literature review confirmed that existing solutions, whether RFID and biometric attendance, IoT-based monitoring, facial-recognition cameras, smart locks or QR-based access control, are either hardware-dependent or limited to a single function. None of them unifies administration, safety, predictive analytics and resource optimization for a hostel.

HostelOS addresses this gap with a layered architecture made up of presentation, application and service, intelligence, data, and notification and security layers. The platform centralizes the management of students, rooms and beds, leave, fees, complaints, mess operations, inventory, maintenance, visitors and emergencies, with role-based access for students, parents or guardians, wardens, security personnel, administrators and service staff. An AI and analytics layer adds complaint classification and priority routing, explainable room allocation, predictive maintenance, meal-demand and inventory forecasting, and risk analysis. An emergency module supports SOS reporting, alerts, acknowledgement tracking and audit logging, supported by a digital map of fire and first-aid equipment.

A key design decision is the exceptional-movement QR workflow. Routine college travel needs no authorization, and QR tokens are issued only for late returns, holiday outings, and medical or personal visits. Each token is generated after staff approval, validated entirely on the server, and logged, while invalid, expired or reused tokens are rejected and recorded. Because the device camera performs the scanning, the

system needs no dedicated RFID, NFC or biometric hardware. This keeps the gate practical and low-cost while exceptional movement stays explicitly authorized and traceable.

Overall, the proposed system moves hostel management from basic record keeping toward intelligent monitoring, prediction, optimization and decision support. It is expected to reduce manual work, improve resource utilization, speed up complaint resolution, reduce food waste through better meal and inventory planning, and strengthen auditable safety response. By choosing a practical, open-source technology stack, the design stays deployable in a typical institutional setting.

5.2 FUTURE SCOPE

The platform can be extended in the following directions:

Training and refining the AI models on real, long-term hostel data, and comparing alternative algorithms to improve accuracy.

Adding offline-capable QR verification and SOS fallback (for example, SMSalerts) for areas with weak connectivity.

Integrating optional IoT sensors, such as smart meters and gas-leak or smoke detectors, to feed real-time data into predictive maintenance and the safety map.

Integrating CCTV analytics for incident verification, with appropriate privacy safeguards.

Connecting HostelOS with the college's student information and fee systems for seamless data exchange.

Adding multilingual support and notification options to reach parents and guardians more effectively.

Scaling the platform to multiple hostels or campuses under a common administrative dashboard.

REFERENCES

[1] K. O. Okokpujie et al., "A Smart Hostel Management System Using RFID and Biometric Technology," *Procedia Computer Science*, 2018.

[2] K. Patel et al., "Hostel Management System with QR," *International Journal of Computer Applications*, 2020.

[3] S. Singh et al., "IoT Based Smart Hostel Management System," in *Proc. 4th International Conference on Intelligent Computing and Control Systems (ICICCS)*, 2020.

[4] S. Bhardwaj et al., "Hybrid Technology Based Smart Hostel Management System Using Artificial Intelligence and Internet of Things," 2022.

[5] A. Jain et al., "Smart Door Access Control System Based on QR Code," *International Journal of Informatics and Communication Technology*, 2023.

[6] S. H. Tan, "Hostel Management System," Final Year Project, Tunku Abdul Rahman University of Management and Technology, 2024.

[7] M. Adedoyin and F. Olukoya, "Development of a Smart Lock System Using QR Code Technology," *ABUAD Journal of Engineering Research and Development*, 2024.

[8] "Smart Hostel Management Through the IoT: Reducing Energy Consumption, Securing Access, and Preventing Theft," IEEE Conference Publication, 2025.

[9] G. Yu et al., "Intelligent Classification of Patient Complaints Using Machine Learning and NLP," *Journal of Medical Internet Research*, 2025.

[10] W. Alsawi and D. M. Ibrahim, "QR Code-Based Access Control Systems: Architectural Taxonomy, Security Landscape, and Future Research Directions," *International Journal of Advanced Computer Science and Applications*, 2026.

[11] "HostelOS: Revised QR Project Document .(v2)," Department project description, [Institution Name].

[12] A. Tharu, K. K. Sah, and H. Kaur, “Design and Implementation of an Integrated Hostel Management System with Role-Based Access for Students and Wardens,” *Journal of Technology Management for Growing Economies*, vol. 16, no. 1, pp. 105–117, 2025.

[13] G. Rajkumar and T. Sivagama Sundari, “Hostel Management System Based on Finger Print Authentication,” *Oriental Journal of Computer Science and Technology*, vol. 11, no. 4, pp. 230–234, 2018. doi: 10.13005/ojcst11.04.08.

[14] Choudhury et al., “Online Hostel Management System,” *ABHIYANTRIKI: An International Journal of Engineering & Technology*, vol. 4, no. 3, pp. 1–6, Mar. 2017.

[15] S. J. Taylor and B. Letham, “Forecasting at Scale,” *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018. doi: 10.1080/00031305.2017.1380080.

[16] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[17] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic Minority Over-sampling Technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[18] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-Based Access Control Models,” *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[19] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, May 2015.

[20] M. T. Ribeiro, S. Singh, and C. Guestrin, “‘Why Should I Trust You?’: Explaining the Predictions of Any Classifier,” in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.